
Position: Mathematical DSL Benchmarks Need Executable Interface Contracts

Anna Genevoux^{* 1} Simon Frieder^{* 2}

Abstract

This position paper argues that *documentation is infrastructure* for reproducible geometry reasoning, and more broadly that mathematical benchmarks built around formal domain-specific languages (DSLs) need an executable interface contract: documented vocabulary should be matched by executable, versioned behavior and runnable examples. We use JGEX (as implemented by Newclid and AlphaGeometry) as a prototypical case study of how documentation–implementation gaps, missing examples, and non-trivial encoding choices can constrain expressivity, affect solver behavior, fragment tool interoperability, and bias benchmark construction. We introduce *JGEXDiv*, a curated collection of 137 Euclidean geometry problems with released JGEX formalizations, natural-language rewrites, and metadata, covering 51.4% of admissible construction-pattern bins under our signature-based analysis. We also release a conformance suite for the documented JGEX interface, including minimal runnable examples, validation scripts, and a predicate-level support matrix. This audit surfaces cross-tool discrepancies, including documented predicates that execute under pinned Newclid v3.0.1 but are not recognized by Yuclid, as well as predicates that are unsupported or unstable under Newclid itself. Finally, we provide an executable NL→JGEX baseline under the same backend. We argue that these artifacts together form an executable interface contract for mathematical DSL benchmarks, and should be treated as first-class research artifacts alongside datasets and evaluation code.

1. Introduction

Despite rapid progress on mathematical reasoning benchmarks, geometry remains comparatively under-resourced with datasets that are simultaneously (i) semantically explicit about objects, constraints, and constructions, (ii) machine-checkable, and (iii) packaged for reproducible benchmarking (Chen et al., 2021; Lu et al., 2021; Chen et al., 2022; Lu et al., 2024; Shi et al., 2025). Most widely used geometry datasets in machine learning are oriented toward multimodal question answering or numerical answers, and therefore do not expose a standardized, prover-compatible construction language or an executable specification of the underlying instance (Chen et al., 2021; Lu et al., 2021; Zhang et al., 2023; Lu et al., 2024).

This scarcity makes it difficult to systematically evaluate automated theorem provers, diagram understanding systems, and neuro-symbolic solvers targeted at geometry reasoning. Recent end-to-end systems show that strong performance on challenging synthetic geometry is possible, but they typically rely on internal symbolic representations that are not intended as public interchange formats, making cross-tool comparisons and dataset reuse difficult (Trinh et al., 2024; Chervonyi et al., 2025). In contrast, formal representations such as JGEX and interchange formats such as I2GATP demonstrate that machine-readable encodings of Euclidean constructions and goals are feasible (Ye et al., 2008; Quaresma, 2012), yet standardized corpora that combine explicit constructions with consistent semantics, executable validation, and benchmark-facing documentation remain uncommon.

At the same time, formalization efforts in interactive theorem proving provide high assurance and reusable libraries for Euclidean geometry (Murphy et al., 2024), but they are primarily designed for expert-driven development rather than serving as lightweight, training-ready benchmarks that can be consumed by multiple geometry tools. Recent work such as LeanGeo (Song et al., 2025) demonstrates that formalizing competition geometry in Lean is feasible, but also highlights substantial formalization and proof-engineering overhead; for dataset-scale benchmarking and natural-language-to-formal translation, it is often cheaper to work directly in an executable construction language such as JGEX, provided that its behavior is documented with runnable, versioned examples. Newer geometry provers

^{*}Equal contribution ¹BenchXiv ²University of Oxford, Oxford, United Kingdom. Correspondence to: Anna Genevoux <anna@benchXiv.org>, Simon Frieder <simon.frieder@cs.ox.ac.uk>.

such as NEWCLID aim to improve accessibility by supporting practical input formats such as JGEX while retaining deductive capabilities (Sicca et al., 2024), which further motivates curating datasets that are both tool-compatible and easy to audit. Geometry is especially illustrative for this broader methodological point because benchmark instances are often executable construction programs rather than mere theorem statements: the same mathematical claim can admit multiple valid formal encodings, and solver behavior can depend materially on which construction graph is exposed. More broadly, this reflects a lesson long emphasized in computational science: much of the practical success of data science has depended not only on headline models and datasets, but on the surrounding reproducibility environment—shared formats, executable artifacts, examples, and documentation that reduce friction and make results reusable by others (Donoho, 2024).

Position: We call the following bundle a *executable interface contract* for mathematical DSL benchmarks: (i) released executable formal instances and validator scripts, (ii) explicit versioning of the toolchain and relevant dependencies, (iii) executable support evidence for the benchmark-used fragment of the formal interface, and (iv) explicit documentation of nontrivial curation decisions such as reformulations, helper constructions, corrections, exclusions, and retained alternative encodings. This contract is the core of what we call a *documentation-as-infrastructure environment*: the benchmark is released together with the executable information needed to re-run, audit, and extend it. Any mathematical benchmark built around a formal DSL that lacks this environment is not fully reproducible. Without it, documentation-implementation gaps and encoding choices silently shape what can be expressed, what runs, and what is actually being measured. These issues matter not only for human users and reviewers, but also for LLM-based formalization pipelines: in a low-resource, version-sensitive DSL, the usable interface is not reliably recoverable from scattered examples alone. We use JGEX geometry as a prototypical illustration of this broader methodological point.

Table 1 previews how these recommendations are currently reflected in representative Lean and executable-geometry benchmarks; Appendix E provides the full evidence and assessment protocol.

Our contributions. We contribute benchmark-critical infrastructure and baseline methodology for mathematical DSL benchmarking, applied here to executable geometry:

- **A solver-audited JGEX corpus (Sections 3.2–3.5):** *JGEXDiv*, a curated collection of 137 Euclidean geometry problems pairing original natural-language state-

ments and sources, JGEX-oriented rewrites that make formalization steps explicit, released JGEX formalizations with run-status annotations under pinned Newclid v3.0.1, and metadata for analysis and filtering. The corpus also retains selected alternative encodings when they serve distinct benchmark purposes.

- **Executable interface evidence and cross-tool inconsistency findings (Sections 3.4 and 3.6; Appendix C):** a predicate-level support matrix over the 33 documented JGEX goal predicates, generated from minimal executable test instances and categorized as supported, unsupported, or unstable under pinned Newclid. The audit surfaces concrete backend inconsistencies: four documented predicates execute end-to-end under pinned Newclid v3.0.1 but are not recognized by Yuclid, while four documented predicates are unsupported and three are unstable under Newclid itself. At the corpus level, the dataset covers all supported goal predicates and $36/70 = 51.4\%$ of admissible construction-pattern bins under our coverage measure.
- **A documented formalization methodology (Section 3.5; Appendix B):** curation principles and worked examples covering alternative encodings, equivalent reformulations when the target relation is not directly expressible, fixed-angle modeling, and the explicit recording of helper constructions and other nontrivial auxiliary choices.
- **An illustrative NL→DSL baseline, instantiated as NL→JGEX (Sections 4–4.3):** an LLM-assisted formalization pipeline for a low-resource geometry DSL, grounded by a compact interface reference card and evaluated by end-to-end executability under pinned Newclid. On the evaluated set, pass@1 executability is 49.5% overall and 89.8% conditional on non-truncated attempts.

Tool-agnostic generalization. Our core claim is not specific to JGEX or Newclid, but to executable benchmarks evaluated against evolving formal interfaces. As Table 1 illustrates, elements of this executable interface contract already appear across the Lean and geometry ecosystems, but their adoption remains uneven. The full artifact-level comparison is given in Appendix E.

2. Related Work

Benchmarking with formal mathematical DSLs spans formal interfaces, learning-oriented datasets, interactive formalization, and end-to-end symbolic systems; geometry makes these issues especially visible.

Formal geometry languages and interchange formats. A foundational line of work develops machine-readable lan-

Table 1. Adherence of representative public benchmark releases to the four components of our proposed executable interface contract: (i) released executable formalizations, (ii) explicit versioning, (iii) executable support evidence for the benchmark-used formal interface, and (iv) documentation of nontrivial curation decisions. Full assessment criteria and supporting evidence are given in Appendix E. “Not found” means that we did not identify corresponding evidence in the reviewed public release.

Benchmark / system	(i)	(ii)	(iii)	(iv)
<i>Lean benchmarks</i>				
LeanEuclid	Yes	Partial	Partial	Partial
ProofNet	Yes	Partial	Partial	Partial
Lean Refactor / Arena	Partial	Yes	Yes	Partial
LeanGeo-Bench	Yes	Yes	Partial	Partial
miniF2F Lean 4	Yes	Yes	Partial	Partial
Formal Conjectures	Yes	Yes	Partial	Yes
Lean Eval	Yes	Yes	Yes	Partial
<i>Executable geometry</i>				
AlphaGeometry2	Partial	Not found	Partial	Partial
TongGeometry	Partial	Partial	Partial	Partial
GenesisGeo	Yes	Partial	Partial	Partial
JGEXDiv (ours)	Yes	Yes	Yes	Yes

guages for Euclidean constructions and conjectures. JGEX, originally designed for the Java Geometry Expert ecosystem, encodes a configuration as a sequence of explicit constructions plus a goal predicate, enabling automated deduction and structured proof exploration (Ye et al., 2008). Complementary efforts aim at interoperability across geometry tools and provers; for example, I2GATP provides an XML-based interchange format to exchange geometric problems and auxiliary data between systems (Quaresma, 2012). These representations show that precise encodings are feasible, but they do not by themselves yield a *benchmark* in the experimental sense: reproducible benchmarking additionally requires a pinned execution target, a validator, and conformance evidence for the fragment of the language actually used.

Learning-oriented geometry datasets.

A second line of work provides multimodal and learning-oriented geometry datasets, including GeoQA (Chen et al., 2021; Cao & Xiao, 2022), Geometry3K (Lu et al., 2021), PGPS9K (Zhang et al., 2023), UniGeo (Chen et al., 2022), MathVista (Lu et al., 2024), and GeoThought (Shi et al., 2025). These datasets support perception and learned reasoning but generally do not expose a prover-checkable construction language with versioned execution semantics, limiting their use for formal-solver or executable NL→formal evaluation.

Interactive theorem proving and verified geometry libraries. A complementary direction formalizes Euclidean geometry inside proof assistants, enabling fully verified proofs and reusable libraries. LeanEuclid (Murphy et al., 2024) develops substantial formal infrastructure for Euclidean geometry. LeanGeo (Song et al., 2025) shows that formalizing competition-style geometry problems at scale

in Lean is feasible, but also highlights the proof-engineering overhead and strong coupling to particular library and tactic ecosystems. For benchmarking and NL→formal translation, these settings reinforce the same methodological requirements we advocate here: version-pinned dependencies, executable artifacts, and explicit interface assumptions about what fragment of the formal language is actually supported by the benchmark toolchain.

Benchmark-facing infrastructure in Lean. Lean benchmarks already instantiate parts of this standard. LeanEuclid and LeanGeo-Bench release reproducible formalizations (Murphy et al., 2024; Song et al., 2025); miniF2F documents corrections to formal statements (Zheng et al., 2021); ProofNet illustrates the versioning problem created by its original Lean 3 release (Azerbaiyev et al., 2023); and Formal Conjectures, Lean Eval, and Lean Refactor provide stronger versioning or executable validation infrastructure (Firsching et al., 2026; Lean Eval Contributors, 2026; Lu et al., 2026). These examples show that executable artifacts and dependency pins are increasingly common, while explicit interface audits and systematic curation records remain less widespread; Appendix E gives the full comparison.

End-to-end solvers and neuro-symbolic systems. Recent systems combine learning with symbolic deduction to solve challenging synthetic geometry problems. AlphaGeometry (Trinh et al., 2024) and AlphaGeometry2 (Chervonyi et al., 2025) demonstrate strong performance on Olympiad-style problems using a neuro-symbolic pipeline. However, these systems typically rely on custom internal representations and pipelines that are not intended as public interchange formats, which complicates cross-tool comparison and reuse of the same instances across provers. Newclid (Sicca et al., 2024) explicitly targets accessibility and interoperability by

supporting practical input formats such as JGEX, making it a natural execution target for an *auditable* JGEX benchmark.

NL→formal translation and low-resource executable targets. Most NL→formal benchmarks target mature languages with large public corpora. JGEX instead has a small, fragmented public footprint and a version-sensitive executable interface, making success depend on learning the effective DSL rather than matching a reference string. This motivates our compact reference card, which constrains generation to supported predicates and constructor signatures.

Positioning and gap. Prior work offers either formal encodings without a curated, solver-validated benchmark corpus, large datasets without an executable formalization, or executable toolchains without benchmark-facing conformance evidence and documented curation choices. Representative Lean and geometry projects increasingly release executable formal artifacts and reproducible software environments, but Appendix E shows that adoption is uneven across recommendations (i)–(iv). Formal instances and dependency manifests are relatively common in Lean, while recent geometry systems increasingly release solver code, formal examples, and generated data. In contrast, benchmark-facing support evidence and systematic records of formalization and curation decisions remain uncommon across both ecosystems. Our contribution targets the still-rare combination of these ingredients in executable geometry: a JGEX corpus paired with natural-language rewrites and packaged with version pinning, end-to-end validation, predicate-level conformance evidence, and worked examples recording nontrivial formalization choices.

3. Methodology

This section describes the construction of *JGEXDiv* as a concrete example of the documentation-as-infrastructure environment advocated above. All artifacts used in this work—including the dataset release, predicate audit, validation scripts, and the NL→JGEX pipeline—are available at <https://github.com/benchxiv/JGEXDiv>.

3.1. Dataset design goals

The dataset is designed to support two complementary uses: (i) reproducible evaluation of automated synthetic-geometry reasoning on formally specified instances, and (ii) natural-language-to-formal translation by pairing each problem with both a natural-language statement and a released JGEX formalization, together with pinned run-status annotations. A central design constraint is interface coverage: we aim to span a broad subset of the documented JGEX predicate vocabulary and construction definitions, while prioritizing benchmark instances that are executable under a pinned solver version in practice and explicitly annotating rows

whose behavior differs across interfaces or versions.

3.2. Corpus and schema

We release a CSV corpus with one row per JGEX instance. Each row pairs an original natural-language statement with a JGEX-oriented rewrite, a released JGEX formalization, source metadata, and run-status annotations under pinned Newclid v3.0.1. A small number of rows behave differently across Newclid interfaces or supporting tools. Instead of removing them, we keep them and record the observed behavior in the run-status and comment fields. Some underlying problems appear in multiple rows when we retain alternative but equivalent encodings that differ in construction granularity or constructor choice; Appendix B.1 gives a representative example. The full field-level schema is documented in the repository.

3.3. Problem sources and selection

We curate problems from multiple sources, including International Mathematical Olympiad (IMO) problems, Euclid-style classical geometry problems, IMO Shortlist problems, and selected textbook-style instances such as classical lemmas and sanity checks. Problems are selected to span diverse construction patterns and goal types and, in particular, to exercise a wide range of documented JGEX predicates and construction definitions. Figure 1 shows the distribution of the current release across these source categories.

In addition to externally sourced problems, we include a set of author-constructed instances designed to increase data diversity and to cover predicates and legacy construction definitions that are underrepresented in contest corpora. For such entries without an external reference, the `statement_source` field uses a short descriptive label prefixed with *ExpertConstructed*.

In the current release, we target broad practical coverage of the usable JGEX vocabulary under pinned Newclid. Concretely, we cover all *supported* goal predicates identified in Section 3.6 and $85/89 \approx 95.5\%$ of the construction definitions. Corpus-level coverage of both goal predicates and construction definitions is summarized in Table 2.

3.4. Coverage analysis

Construction-pattern coverage. To assess how broadly the dataset spans *types of geometric objects*, we measure coverage over construction tokens recorded in the structured metadata field `NL_construction_in_statement`. We track six construction types: `bisector`, `circle`, `line`, `polygon` (grouping all specific polygons under `polygon`), `ray`, and `segment`. We treat `point` as implicit and exclude it from the signature space, since every non-degenerate configuration presupposes points.

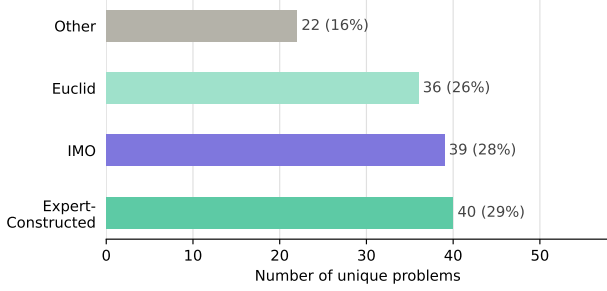


Figure 1. Distribution of the 137 problems in *JGEXDiv* by source category.

Not all subsets of these six tokens correspond to meaningful Euclidean configurations, so we enforce the implications `polygon` $\Rightarrow$ `segment` and `bisector` $\Rightarrow$ `line`. After applying these constraints, there are 35 valid construction signatures. We report coverage separately for problems with and without explicit numerical content (`numerical_concept` $\in$ {yes, no}), yielding 70 possible (`numerical_concept`, signature) bins. In the current release, 36 bins occur at least once, giving construction-pattern coverage $36/70 = 0.5143$ (51.4%). We export the full observed/missing bin table as `construction_coverage.csv` and provide a script to reproduce the computation.¹

Corpus-level coverage. Table 2 summarizes key coverage quantities for *JGEXDiv*. Two aspects are particularly important for reproducibility. First, the dataset is restricted to goals that are *executable* under pinned Newclid v3.0.1, rather than the full documented predicate vocabulary. Second, we report construction-definition coverage separately, since definition coverage affects what configurations can be expressed even when the goal predicates are supported. Four construction definitions (`test_r20`, `test_r25`, `e5128`, and `3peq`) were deliberately excluded from the dataset because each is a one-off definition introduced for a single specific problem instance rather than a reusable geometric construction. These two coverage measures capture different notions of breadth: construction-definition coverage tracks how much of the documented constructor vocabulary appears at least once, whereas construction-pattern coverage tracks how widely the corpus spans problem types under our coarse statement-level schema.

3.5. Formalization and curation decisions

For each problem, we produce two natural-language forms: the `original_NL.problem.statement` (verbatim) and the `NL-JGEX.problem.statement` rewrite.

¹https://github.com/benchxiv/JGEXDiv/tree/main/evaluation/compute_construction_coverage.py

Table 2. Corpus-level statistics and interface coverage under pinned Newclid v3.0.1.

Quantity	Value
Total problems (unique <code>problem_id</code>)	137
Pinned solver version for executability	Newclid v3.0.1
Documented JGEX goal predicates audited	33
Supported predicates (end-to-end executable)	26 ($\approx 78.8\%$)
Unsupported predicates (not executable)	4 ($\approx 12.1\%$)
Unstable predicates (no accessible minimal example)	3 ($\approx 9.1\%$)
Legacy construction definitions covered	62/64 ($\approx 96.9\%$)
New construction definitions covered ²	23/25 (= 92.0%)
Total construction definitions covered	85/89 ($\approx 95.5\%$)

²“New construction definitions” refers to the newer JGEX definition set documented in the Newclid manual: https://newclid.github.io/Newclid/manual/building_a_problem_setup/jgex/definitions.html#new-definitions.

The rewrite serves two purposes: (i) it resolves ambiguity in ordinary geometry prose, such as implicit intersections, unnamed lines, or “let” steps without explicit construction constraints, and (ii) it acts as a pedagogical bridge by mirroring the explicit, stepwise nature of JGEX.

We then translate the rewritten statement into an executable `JGEX.problem.statement`. A single informal theorem may admit multiple valid JGEX encodings. In selected cases, we retain alternative encodings when they serve different benchmark purposes: an expanded encoding may build an object from first principles and expose a richer dependency graph, whereas a compact encoding may use a direct constructor such as `circumcenter`. These alternatives preserve the underlying mathematical claim while changing construction granularity and, consequently, proof-search behavior under synthetic solvers. Appendix B.1 illustrates this phenomenon on Euclid IV.5.

Conjunctive informal conclusions are encoded directly as multiple JGEX goals whenever the required predicates are supported. We therefore treat multi-goal instances as normal JGEX usage rather than as a special workaround; Appendix B.2 provides a representative fixed-angle example.

When the original target claim cannot be expressed directly using the supported predicate vocabulary, we use one of two standard strategies:

- **Equivalent reformulation:** replace the target claim with a logically equivalent claim expressible in JGEX, for example by rewriting an area equality into a ratio statement involving a shared base and corresponding heights. See Appendix B.3.

- **Documented helper constructions:** introduce auxiliary objects only when their role is geometrically interpretable and explicitly recorded—either because they are needed to express the intended claim with the available interface, or because they expose a standard intermediate relation that materially affects solver behavior without changing the formal goal. See Appendix B.2.

Because benchmark control and interpretability are central goals of the dataset, we do not automatically adopt every executable JGEX string found in prior solver-oriented corpora as a canonical formalization. Some external encodings contain auxiliary constructions that may help proof search but whose geometric role is not evident from the statement or from a recognizable synthetic proof pattern. Unless we can justify such additions, we omit them, replace them with a simpler readable encoding, or retain them only as non-canonical variants with an explicit comment.

We document all nontrivial reformulations, auxiliary choices, and alternative encodings in the `comment` field and provide worked examples in Appendix B.

3.6. Predicate-level audit and executable validation

A key practical challenge is that documented JGEX goal predicates are not uniformly usable in practice under Newclid: some are missing end-to-end implementations, some fail during problem building, and some cannot be reliably evaluated because the documentation does not provide an accessible minimal example. To characterize this gap systematically, we audit the full set of 33 documented goal predicates using minimal JGEX test instances designed to isolate predicate semantics. More generally, this kind of audit should be standard for any mathematical DSL benchmark whose documented interface is interpreted by an evolving backend.

For each predicate, we record whether it is accepted syntactically and whether it executes end-to-end under pinned Newclid v3.0.1. We additionally note Yuclid acceptance where relevant as supplementary evidence about documentation and compatibility gaps, but Newclid executability determines benchmark inclusion. For the purposes of this dataset, a predicate is *supported* if a minimal instance builds and runs end-to-end in Newclid v3.0.1, *unsupported* if it does not, and *unstable* if no reliable accessible minimal test instance can be constructed from the documentation and available resources. In our audit, 26 predicates are supported, 4 are unsupported, and 3 are unstable. The audit also shows that JGEX support is not backend-independent: four documented predicates (`DIFFERENT`, `N_COLLINEAR`, `N_PERPENDICULAR`, and `N_PARALLEL`) execute end-to-end under pinned Newclid v3.0.1 but are not recognized

by Yuclid, reinforcing our central point that documentation alone does not determine the effective benchmark interface.

At the instance level, each row is annotated with `problem_runs_on_Newclid_3_0_1`. This flag records *executability*: the JGEX instance must parse, build, and launch solver execution without raising an exception under the pinned version. A successful proof and a timeout are both treated as executable outcomes; build-time or runtime exceptions are not. This criterion matches the paper’s emphasis on executable, versioned interfaces rather than proof success alone.

Full per-predicate evidence is reported in Appendix C, including the exact minimal syntax tested, Yuclid acceptance, Newclid v3.0.1 outcome, and the observed failure mode where applicable. This appendix table serves two purposes. First, it guides dataset curation by restricting benchmark goals to predicates that are supported under Newclid v3.0.1. Second, it improves reproducibility by grounding interface claims in executable, versioned test cases rather than manual inspection alone. A minimal reproducible case study is discussed in Appendix D.

3.7. Threats to validity

Two threats are worth making explicit. First, executability is version-dependent: predicate support and constructor behavior can change across Newclid releases, so results should be interpreted as tool- and version-specific unless revalidated under the same pins. Second, restricting benchmark goals to supported predicates can bias the dataset toward relations that are currently implemented. We mitigate this by (i) publishing a complete predicate audit with minimal executable test cases, and (ii) retaining the original natural-language statements together with explicit NL rewrites so instances can be re-targeted as the interface expands. To prevent silent drift, we recommend (and structure our artifacts to support) re-running the conformance suite in CI across a small Newclid version matrix and publishing per-version support deltas.

3.8. Pedagogical material

We include a short JGEX tutorial and worked appendix examples covering alternative encodings, fixed-angle constructions, equivalent reformulation, and documented helper constructions.

4. From Natural Language to a Low-Resource Geometry DSL

As a broader methodological recommendation, benchmarks targeting formal DSLs should evaluate NL→DSL generation under the same pinned backend used for the benchmark.

We illustrate this with JGEX, a niche, version-sensitive geometry DSL with a small public footprint. The resulting NL→JGEX pipeline serves both as a baseline and as a stress test of whether the documented benchmark interface is learnable and executable in practice.

Reference card. In preliminary prompting, the model often produced geometrically plausible but non-executable JGEX, including hallucinated predicates, incorrect constructor arities, and near-miss syntax. Because public examples are sparse and do not consistently distinguish documented vocabulary from the interface executable under a specific Newclid version, we provide a compact reference card with supported predicates, common construction patterns, canonical signatures, and minimal examples. It acts as a lightweight specification of the pinned interface rather than solving the task for the model.

4.1. Pipeline overview

Given a natural-language problem statement s , we run a simple propose–validate pipeline:

1. **Prompting:** query the LLM with (i) an instruction to output only JGEX, (ii) the reference card, and (iii) $k = 20$ in-context examples consisting of paired NL statements and correct JGEX formalizations.
2. **Extraction:** extract a candidate JGEX program from the response, if any.
3. **Executable validation:** attempt to build and run the candidate with pinned Newclid v3.0.1.

We record three outcome types: *Executable*, *API-truncated* (`finish_reason=length` and no emitted program text), and *Other failure* (e.g., no extractable JGEX or a build/run failure under Newclid). Figure 2 summarizes the pipeline.

4.2. Implementation details

We query `openai/gpt-5.4-pro` via OpenRouter using the Chat Completions API with `temperature = 0`, `top-p = 1`, and `max_tokens=3000`. We do not set an explicit reasoning-effort override in the request; all results therefore use the provider default reasoning setting for this model on OpenRouter. Validation uses Newclid’s `JGEXProblemBuilder` followed by `GeometricSolverBuilder`. Our criterion is *end-to-end executability*: the instance must parse, build, and run without raising an exception.

Executability is intentionally weaker than semantic correctness. It does not certify that the generated JGEX is fully faithful to the original NL statement or that the stated goal is

mathematically appropriate; it certifies only that the output is a runnable formal artifact under the pinned version. This choice matches the paper’s focus on executable, versioned interfaces: for a niche DSL, many failures arise before proof search even begins.

The full prompt construction, including the reference card and the curated set of $k = 20$ examples, is implemented in `pipeline.py`. We log prompts, raw responses, extracted candidates, and Newclid exceptions for reproducibility.

4.3. Results

Evaluation set. We reserve 20 problems for in-context examples and evaluate on the remaining 117. For the analyses in this section, we further restrict to the subset whose human reference encoding (`kgex_reference`) itself builds and runs under pinned Newclid v3.0.1. This removes 12 rows with broken references and yields 105 unique problems (107 dataset rows because two `problem_ids` have multiple encodings). This restriction isolates the NL→JGEX formalization bottleneck from failures already present in the oracle reference.

Pass@1 executability. Table 3 reports `pass@1` end-to-end executability on the evaluated set. Overall, $53/107 = 49.5\%$ of attempts produce an executable JGEX formalization. Failures are dominated by API truncation: $48/107 = 44.9\%$ terminate with `finish_reason=length` and return no JGEX content. The remaining $6/107 = 5.6\%$ are non-truncated failures, such as responses with no extractable program or candidates that fail during Newclid build/run. Conditioning on non-truncated attempts, the executability rate rises to $53/59 = 89.8\%$.

A useful way to read Table 3 is to separate *generation-budget failure* from *formalization failure*. The raw source-level gap is driven largely by truncation: IMO rows have the lowest `pass@1` executability, but also by far the highest truncation rate. Conditioning on rows for which the API returns a non-truncated candidate, executability is high across all source categories. This suggests that, under our prompt setup, the dominant bottleneck is often obtaining a complete candidate within the API budget rather than mapping returned text into runnable JGEX once generation completes.

Truncation mitigation (limited probe). Because API truncation is the dominant failure mode, we ran a small follow-up probe on the 48 previously truncated rows using the same prompt but adjusting only `reasoning_effort="low"` (keeping `max_tokens=3000`). This recovered 9 executable JGEX programs (9/48); the remaining 39 again terminated with `finish_reason=length` and `content=None`. Recovery was concentrated outside IMO (IMO: 0/28, ExpertConstructed: 6/12, Euclid: 2/6, Oth-

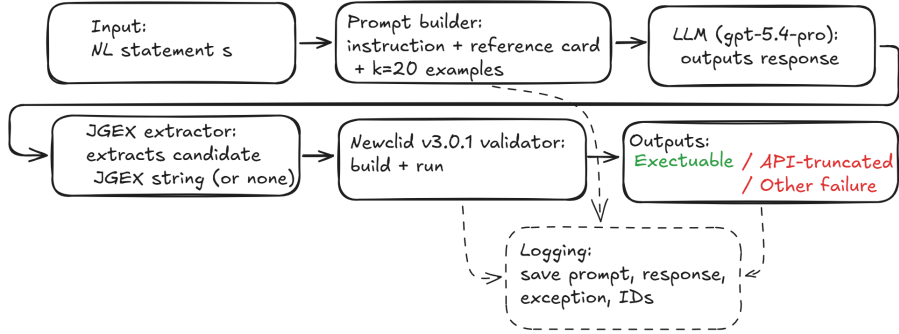


Figure 2. NL→JGEX propose–validate pipeline. A natural-language statement is paired with reference card and $k = 20$ in-context examples, then passed to GPT-5.4 Pro. The extracted candidate JGEX program is validated by building and running it under pinned Newclid v3.0.1. Outcomes are categorized as *Executable*, *API-truncated*, or *Other failure*.

Table 3. Pass@1 end-to-end executability by source on the evaluated set (107 dataset rows). API truncation denotes `finish_reason=length` with `content=None` (no JGEX program returned).

Source	N	Exec. count	Exec. rate	API trunc. count	API trunc. rate	Other fail
Euclid	29	21	0.724	6	0.207	2
IMO	32	3	0.094	28	0.875	1
Other/meta	7	4	0.571	2	0.286	1
ExpertConstructed	39	25	0.641	12	0.308	2
All	107	53	0.495	48	0.449	6

er/meta: 1/2). This suggests that truncation on long, complex statements (notably IMO) is robust under modest decoding changes; we leave a systematic study of prompt-length trade-offs (e.g., varying k) to future work. The rerun script is provided in `rerun.truncated.py`.

What this result measures. These numbers measure executable NL→JGEX formalization under a fixed backend, not standalone geometry-solving ability. Newclid is used only as the pinned validator/backend once a JGEX program has been produced.

5. Alternative Views

Our position is that mathematical benchmarks built around executable DSLs should be released with (i) executable formalizations, (ii) explicit versioning, (iii) executable support evidence for the benchmark-used fragment of the formal interface, and (iv) documented curation decisions such as reformulations, helper constructions, corrections, exclusions, and alternative encodings. Geometry, and JGEX in particular, provides a useful prototype case because these issues are unusually visible there: benchmark instances are executable construction programs, interface support is version-sensitive, and encoding choices can materially affect solver behavior.

View 1: The codebase is the specification; extra documentation and predicate audits are unnecessary. A common engineering view is that implementation is the

only authoritative specification: if a predicate or constructor exists, its meaning is defined by the code path that parses and executes it. Under this view, a pinned commit hash and runnable inputs suffice, and extra documentation risks drift. A related argument is that unsupported predicates will simply fail at runtime, after which users can revise the affected instances. The counterpoint in a benchmarking setting is that “read the code” and runtime-only discovery do not scale as usability or reproducibility strategies. Predicate semantics are typically distributed across parsing, problem building, and solver internals, and end-to-end support can fail even when a feature is documented. Different users may therefore end up evaluating different working subsets depending on solver version, local patches, or incidental modeling choices. A lightweight conformance layer—minimal runnable examples, version-pinned support tables, and explicit curation notes—makes the executable interface auditable without attempting to replace the codebase.

View 2: Modern LLMs can infer a niche DSL from a few examples; curated documentation is redundant. A more recent view is that frontier LLMs can serve as the interface layer: given a handful of examples, they can infer syntax, explain undocumented features, and generate working code on demand. In that framing, maintaining tutorials, support matrices, or reference cards is unnecessary overhead. Our NL→JGEX experiments suggest a more cautious conclusion. For a low-resource geometry DSL, scattered ex-

amples alone are not a stable specification: without explicit interface guidance, the model often produces near-miss syntax, wrong arities, or hallucinated predicates. A compact, version-pinned reference card and minimal runnable examples provide a deterministic anchor for both humans and models.

View 3: If an encoding runs, its internal construction choices do not matter. One might argue that benchmark quality is determined only by executability: if two JGEX encodings state the same theorem and both run, their internal construction choices are irrelevant. In practice, however, alternative encodings can materially change the construction graph and therefore proof-search behavior. Building an object “from first principles” versus invoking a direct constructor, or adding an auxiliary helper point versus omitting it, changes what intermediate relations are explicit and what the solver is effectively being asked to do. For this reason, benchmark curation should document nontrivial reformulations, helper constructions, and retained alternative encodings rather than treating all executable strings as interchangeable.

View 4: Restricting to supported predicates and interpretable encodings biases the benchmark. A final concern is that curating around supported predicates and human-interpretable encodings may bias the benchmark toward relations that are currently implemented or toward cleaner formalizations than those used in solver-oriented corpora. This is a real trade-off. Our choice prioritizes executability, auditability, and controlled comparison. We mitigate the resulting bias by (i) publishing unsupported and unstable predicates explicitly, (ii) preserving the original natural-language statements and JGEX-oriented rewrites so instances can be re-targeted as the interface expands, and (iii) documenting non-canonical helpers and alternative encodings when they materially affect solver behavior.

6. Call to Action

We recommend that mathematical DSL benchmarks ship an executable interface contract comprising the four components above. A benchmark should ship not only formal instances, but also the executable interface environment needed to rerun, audit, and extend them.

Minimum requirements.

Release executable formal instances together with validator scripts; explicitly version the toolchain and relevant dependencies; provide executable support evidence for the benchmark-used formal interface; and document nontrivial curation decisions, including reformulations, helper constructions, corrections, exclusions, and alternative encodings.

Best practices. Also publish versioned support audits for predicates and constructors; provide stable documentation with runnable examples and a compact reference card; preserve both original natural-language statements and DSL-oriented rewrites when possible; and routinely evaluate NL→DSL pipelines under the same pinned backend.

Reporting and review. NL→formal evaluations should report model, reasoning, prompt, extraction, and backend details and distinguish formalization failures from downstream proving failures. Coverage claims should be backed by auditable evidence, and benchmark submissions should be judged using executable validation rather than surface plausibility alone.

7. Conclusion

We argued for a general position about mathematical benchmarks built around executable DSLs: reproducibility requires a documentation-as-infrastructure environment, not just released instances and evaluation code. Concretely, this means executable formal instances, explicit versioning, support evidence for the benchmark-used interface, validators and runnable examples, and documentation of nontrivial curation choices. We used Euclidean geometry in JGEX/Newclid as a case study and introduced *JGEXDiv*, a curated corpus of 137 geometry problems paired with released JGEX formalizations, JGEX-oriented natural-language rewrites, and pinned run-status annotations.

A central contribution is the executable interface evidence accompanying the corpus. Our predicate-level audit covers the documented JGEX goal predicates under pinned Newclid and exposes gaps between documented vocabulary and practical usability. Together with construction coverage and version-pinned validation scripts, this makes the benchmark-used fragment of JGEX an auditable executable contract rather than an implicit working subset discovered only at runtime.

We also evaluated a simple NL→JGEX baseline for this low-resource DSL. Using GPT-5.4 Pro with a compact interface reference card and pinned Newclid validation, the pipeline achieves 49.5% pass@1 end-to-end executability, rising to 89.8% conditional on non-truncated attempts. This should be interpreted as evidence about the formalization bottleneck rather than a head-to-head comparison between the LLM and Newclid as geometry reasoners.

The dataset remains limited by version-dependent executability, incomplete construction-pattern coverage, and an NL→JGEX evaluation that measures executability rather than semantic faithfulness. Future work includes expanding coverage, improving automated consistency checks, and separating formalization loss from downstream proving loss more directly.

Acknowledgements

We thank the Newclid team for helpful answers and clarifications during the early stages of this project. We are also grateful to Philip Vonderlind for valuable help with code coverage and for insightful discussions on the machine learning aspects of this work.

References

- Azerbaiyev, Z., Piotrowski, B., Schoelkopf, H., Ayers, E. W., Radev, D., and Avigad, J. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics, 2023. URL <https://arxiv.org/abs/2302.12433>.
- Cao, J. and Xiao, J. An augmented benchmark dataset for geometric question answering through dual parallel text encoding. In *Proceedings of the 29th International Conference on Computational Linguistics*, pp. 1511–1520. International Committee on Computational Linguistics, 2022. URL <https://aclanthology.org/2022.coling-1.130/>.
- Chen, J., Tang, J., Qin, J., Liang, X., Liu, L., Xing, E. P., and Lin, L. GeoQA: A geometric question answering benchmark towards multimodal numerical reasoning. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021. URL <https://aclanthology.org/2021.findings-acl.46/>.
- Chen, J., Li, T., Qin, J., Lu, P., Lin, L., Chen, C., and Liang, X. UniGeo: Unifying geometry logical reasoning via reformulating mathematical expression, 2022. URL <https://arxiv.org/abs/2212.02746>.
- Chervonyi, Y., Trinh, T. H., Olšák, M., Yang, X., Nguyen, H., Menegali, M., Jung, J., Verma, V., Le, Q. V., and Luong, T. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2, 2025. URL <https://arxiv.org/abs/2502.03544>.
- Delta Lab AI. Lean refactor arena. Hugging Face Spaces, 2026. URL <https://huggingface.co/spaces/delta-lab-ai/lean-refactor-arena>.
- Donoho, D. Data science at the singularity. *Harvard Data Science Review*, 6(1), 2024. doi: 10.1162/99608f92.b91339ef. URL <https://hdsr.mitpress.mit.edu/pub/g9mau4m0>.
- Firsching, M., Lezeau, P., Mercuri, S., Horváth, M. Z., Dillies, Y., Sönne, C., Wieser, E., Zhang, F., Hubert, T., Agüera y Arcas, B., and Kohli, P. Formal conjectures: An open and evolving benchmark for verified discovery in mathematics. *arXiv preprint arXiv:2605.13171*, 2026. URL <https://arxiv.org/abs/2605.13171>.
- Lean Eval Contributors. Lean Eval: Comparator-based Lean formal mathematics evaluation. GitHub repository, 2026. URL <https://github.com/leanprover/lean-eval>.
- Lu, J., Kong, S., Stehling, R., Yang, K., Wang, Z., Sun, W., and Chen, W. Lean refactor: Multi-objective controllable proof optimization via agentic strategy search. *arXiv preprint arXiv:2605.20244*, 2026. URL <https://arxiv.org/abs/2605.20244>.
- Lu, P., Li, X., Tang, J., and Hu, R. Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021. URL <https://aclanthology.org/2021.acl-long.528/>.
- Lu, P., Bansal, H., Xia, T., Liu, J., Li, C., Hajishirzi, H., Cheng, H., Chang, K.-W., Galley, M., and Gao, J. Math-vista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations (ICLR)*, 2024.
- Murphy, L., Yang, K., Sun, J., Li, Z., Anandkumar, A., and Si, X. Autoformalizing Euclidean geometry, 2024. URL <https://arxiv.org/abs/2405.17216>.
- Quaresma, P. An XML format for conjectures in geometry, 2012. URL <https://arxiv.org/abs/1207.2511>.
- Shi, N., Qin, C., Song, S., and Luo, M. GeoThought: A dataset for enhancing mathematical geometry reasoning in vision-language models, 2025. URL <https://arxiv.org/abs/2510.21881>.
- Sicca, V., Xia, T., Fédérico, M., Gorinski, P. J., Frieder, S., and Jui, S. Newclid: A user-friendly replacement for AlphaGeometry, 2024. URL <https://arxiv.org/abs/2411.11938>.
- Song, C., Wang, Z., Pu, F., Wang, H., Lin, X., Liu, J., Li, J., and Liu, Z. LeanGeo: Formalizing competition geometry problems in Lean, 2025. URL <https://arxiv.org/abs/2508.14644>.
- Trinh, T. H., Wu, Y., Le, Q. V., et al. Solving olympiad geometry without human demonstrations. *Nature*, 625: 476–482, 2024.
- Ye, Z., Chou, S.-C., and Gao, X.-S. An introduction to Java geometry expert. In *Proceedings of the 7th International Workshop on Automated Deduction in Geometry (ADG 2008)*, Lecture Notes in Computer Science, pp. 189–195. Springer, 2008.

Zhang, C., Song, J., Li, S., Liang, Y., Ma, Y., Wang, W., Zhu, Y., and Zhu, S.-C. Proposing and solving olympiad geometry with guided tree search, 2024. URL <https://arxiv.org/abs/2412.10673>.

Zhang, M.-L., Yin, F., and Liu, C.-L. A multi-modal neural geometric solver with textual clauses parsed from diagram, 2023. URL <https://arxiv.org/abs/2302.11097>.

Zheng, K., Han, J. M., and Polu, S. minif2f: A cross-system benchmark for formal olympiad-level mathematics, 2021. URL <https://arxiv.org/abs/2109.00110>.

Zhu, M., Wang, Z., Ji, S., Du, Z., Ke, J., Deng, X., Yin, Z., Huang, X., Wang, H., and Chen, W. Genesisgeo: Technical report, 2025. URL <https://arxiv.org/abs/2509.21896>.

A. JGEX Quick Tutorial

To use Newclid, each geometry problem must first be written in the JGEX format. This appendix provides a short tutorial explaining the basic structure of a JGEX file and how to describe a geometric construction and its goal step by step.

JGEX is the language Newclid uses to represent a geometry problem in a structured way. It converts the natural-language description of a figure into explicit constructions and a goal that the system can check. JGEX covers the standard objects and relations of Euclidean geometry, but some problems cannot be expressed if they rely on concepts outside this vocabulary.

A JGEX file consists of a sequence of constructions that build the objects of the problem, optionally followed by helper constructions, and ends with a goal line that specifies what to prove. The definitions of all available constructors and predicates can be found in the online manual at newclid.github.io/Newclid/manual/building-a-problem-setup/jgex/databases/index.html.

A minimal example

Here is a small JGEX script showing a triangle, a point on a side, and the goal of proving collinearity:

```
a b c = triangle a b c; p = on_line p b c ? coll p b c
```

This example illustrates the basic pattern: constructions that define the figure, followed by a single goal to be verified by Newclid.

1. Constructions

Each construction has the form

`name = constructor args`

Names can be chosen freely and are usually short labels. The number of names on the left must match the number of objects created by the constructor.

Multiple constructions are separated by semicolons `;`. A single construction may contain several conditions, which are separated by commas `,` when the object is defined by more than one requirement.

Example 1 (triangle):

```
a b c = triangle a b c
```

This creates a triangle ABC .

Example 2 (midpoint and circle):

```
a b = segment a b; o = midpoint o a b; c = on_circle c o a
```

Here AB is a segment, O is its midpoint, and C lies on the circle centered at O with radius OA .

Example 3 (intersection defined by two conditions):

```
z = on_line z x y, on_line z b c
```

Here, the comma means that Z satisfies both conditions: it lies on line XY and on line BC . So Z is the intersection point of the two lines.

2. Helper constructions

If the problem needs extra elements to make the configuration complete, they are written after a vertical bar `|`. Helper constructions follow the same format as regular constructions. They are not part of the original problem statement but are added to help JGEX process the goal. For example, one may introduce additional points to construct a line or direction that is needed to express the final predicate.

Example:

```
a b c = triangle a b c;  
d e f i = incenter2 d e f i a b c;
```

```

...
t = on_line t p q, on_line t i d
| n = orthocenter n c a i;
  g = on_pline g c a b, on_pline g a c n
? perp a t a i

```

Here the points n and g do not appear in the original problem. They are helper constructions added so that JGEX can explicitly form a line through A perpendicular to AI , allowing the system to check the goal `perp a t a i`. A full explanation of this example is given later in *A Walk Through some examples: An Example to show the utility of helper constructions*.

3. Goal line

The goal line has the form

```
? predicate args
```

It starts with a question mark `?` and is followed by a predicate such as `coll`, `cong`, `cyclic`, or `eqangle`. It expresses what needs to be proved. Multiple goals are separated by semicolons `;`.

Example 1 (collinearity):

```
? coll p b c
```

This states that the points P , B , and C are collinear.

Example 2 (segment equality):

```
? cong p q q r
```

This states that the segments PQ and QR are equal in length.

Example 3 (cyclicity):

```
? cyclic p q r m
```

This states that the four points P , Q , R , and M lie on the same circle.

Example 4 (angle equality):

```
? eqangle e c e j e j e f
```

This states that $\angle ECEJ = \angle EJE F$, i.e., two directed angles are equal.

4. Minimal Template

This template summarizes the structure of a JGEX file.

```

⟨jgex-file⟩ ::= ⟨constructions-section⟩⟨goal-section⟩
⟨constructions-section⟩ ::= # CONSTRUCTIONS ⟨main-construction⟩ ⟨helper-construction-list⟩
⟨main-construction⟩ ::= ⟨construction-line⟩
⟨helper-construction-list⟩ ::= ⟨helper-construction⟩⟨helper-construction-list⟩ | ε
⟨construction-line⟩ ::= ⟨name⟩ = ⟨constructor⟩ ⟨args⟩
⟨helper-construction⟩ ::= | ⟨name⟩ = ⟨constructor⟩ ⟨args⟩
⟨goal-section⟩ ::= # GOAL ⟨goal-line⟩
⟨goal-line⟩ ::= ? ⟨predicate⟩ ⟨args⟩.

```

B. A Walk Through Some Examples

The examples below illustrate three recurring phenomena in the dataset: alternative encodings of the same theorem, direct encoding of fixed-angle and conjunctive conclusions, and equivalent reformulation when the target relation is not directly supported. We intentionally restrict the walkthrough to formalizations whose main constructions have a clear geometric interpretation; opaque solver-specific scaffolding inherited from external corpora is not used as a canonical example.

B.1. Euclid IV.5: two formulations of the same problem

This example shows that the same mathematical claim can be represented by distinct JGEX encodings. Both dataset rows target the same goal, but they differ in how the key object—the circumcenter—is constructed. The two rows correspond to distinct `problem_ids` in the dataset.

Geometric statement. To circumscribe a circle about a given triangle.

Common goal.

```
? circle o a b c
```

Formulation 1: explicit construction.

```
a b c = triangle a b c;
d = midpoint d a b;
e = midpoint e a c;
u = on_tline u d a b;
v = on_tline v e a c;
o = intersection_ll o d u e v;
```

Here the circumcenter is built “from first principles” in the Euclidean style: D and E are midpoints, DU and EV are perpendiculars to the corresponding sides, and O is their intersection. This formulation exercises basic constructors (`midpoint`, `on_tline`, `intersection_ll`) and makes the intended reasoning steps explicit in the construction graph.

Formulation 2: direct constructor.

```
a b c = triangle a b c;
o = circumcenter o a b c;
```

This encoding introduces O in a single step via the `circumcenter` constructor, yielding a minimal construction whose purpose is to test end-to-end support for that constructor. Such compact encodings are also useful inside larger Olympiad-style formalizations, where the circumcenter is only an intermediate object and re-deriving it from first principles would add low-value construction overhead.

Why both formulations are useful in the dataset. Both rows target the same predicate `? circle o a b c`, but they serve different evaluation goals. The explicit formulation stresses solver behavior on a richer dependency structure and provides construction-pattern coverage. The direct formulation provides constructor-level conformance evidence: it checks that `circumcenter` is correctly implemented and interoperates with downstream predicates (here, `circle`). More generally, the pair illustrates how the same mathematical claim can be represented at different levels of construction granularity, with corresponding differences in proof-search difficulty.

B.2. IMO 1975 Problem 3: encoding fixed angles and conjunctive goals

This example illustrates how numerical angle constraints from a contest problem can be encoded in JGEX using `s_angle`. It also shows a typical pattern where a single instance contains multiple goals (here, a perpendicularity and a segment congruence), together matching the original two-part conclusion.

Geometric statement. On the sides of an arbitrary triangle ABC , triangles ABR , BCP , CAQ are constructed externally with $\angle CBP = \angle CAQ = 45^\circ$, $\angle BCP = \angle ACQ = 30^\circ$, $\angle ABR = \angle BAR = 15^\circ$. Prove that $\angle QRP = 90^\circ$ and $QR = RP$.

JGEX construction.

```

a b c = triangle a b c;
p = s_angle c b p 45o, s_angle b c p 150o;
q = s_angle a c q 30o, s_angle c a q 135o;
r = s_angle b a r 15o, s_angle a b r 165o
| t = eq_triangle t b r;
? perp q r r p; cong q r r p

```

Why this encoding matches the original statement. The predicate `perp q r r p` asserts $QR \perp RP$, i.e. $\angle QRP = 90^\circ$, and `cong q r r p` asserts $QR = RP$.

Fixed angles are expressed using `s_angle`: for example, `s_angle c b p 45o` constructs P so that the directed angle $\angle CBP$ is 45° . To enforce that the three attached triangles are *external*, we encode one of the vertex angles using its supplement (e.g. 150° rather than 30°), which selects the configuration on the opposite side of the corresponding base line.

Helper construction. In one solver-oriented variant, we add the auxiliary point T by constructing an equilateral triangle on BR . This helper is not needed to state the problem, but it makes an explicit 60° relation available in the construction graph, a pattern that appears in classical synthetic solutions. Because our dataset prioritizes interpretable formalizations, such helpers are included only when their geometric role can be explained.

B.3. ExpertConstructed example: encoding an area equality without an area predicate

This ExpertConstructed example illustrates a common reformulation pattern. Since JGEX has no predicate for triangle area, an equality of areas can be rewritten into a length-ratio statement involving a base and a corresponding height.

Geometric statement (informal). Let P, Q, K be non-collinear, and let R be the midpoint of segment PQ . Let K_1 be the perpendicular foot of K on line PQ . Choose a point L_1 on line PQ , and construct L so that $LL_1 \perp PQ$ and $|LL_1| = |KK_1|$. Prove that triangles RPK and RQL have equal area.

JGEX construction. We construct R as the midpoint of PQ . The point K_1 is the perpendicular foot of K on PQ , so $|KK_1|$ is the height of $\triangle RPK$ to the base line PQ . We then choose L_1 on PQ and construct L on the perpendicular through L_1 such that $|LL_1| = |KK_1|$:

```

p q k = triangle p q k;
r = midpoint r p q;

k1 = foot k1 k p q;      // K1 is foot from K to PQ, so KK1 is the height
l1 = on_line l1 p q;     // choose L1 on PQ (foot point for L)
l = on_tline l l1 p q, eqdistance l l1 k k1; // L on perp through L1 to PQ and L1L = KK1
? eqratio k k1 l l1 r q r p

```

Why the goal encodes area equality. Because P, Q, R are collinear, the bases RP and RQ lie on the same line PQ . Moreover, by construction, $KK_1 \perp PQ$ and $LL_1 \perp PQ$, so $|KK_1|$ and $|LL_1|$ are the corresponding heights from K and L to the base line PQ . Thus

$$[\triangle RPK] = \frac{1}{2} |RP| |KK_1| \quad \text{and} \quad [\triangle RQL] = \frac{1}{2} |RQ| |LL_1|.$$

Therefore $[\triangle RPK] = [\triangle RQL]$ is equivalent to

$$\frac{|KK_1|}{|LL_1|} = \frac{|RQ|}{|RP|},$$

which is exactly what the JGEX predicate `eqratio k k1 l l1 r q r p` asserts.

C. Predicate support evidence

This appendix reports script-generated predicate-level usability evidence for documented JGEX predicates. For each predicate, our audit scripts run a minimal JGEX test case and record two outcomes: (i) whether Yuclid accepts the predicate (i.e., recognizes/parses the statement), and (ii) whether the same predicate executes end-to-end in Newclid v3.0.1 (i.e., the problem builds and runs without build-time or runtime errors).

These results are used in Section 3.6 to select a usable target vocabulary for the benchmark. The table is generated from a fixed suite of minimal JGEX test cases; the generation script can be found in the repository under the name `build_predicate_support_matrix.py`

How to read Table 4. *Yuclid* indicates whether the predicate is recognized/parses. *Newclid 3.0.1* indicates whether the minimal test case runs end-to-end. The *Notes* column summarizes the observed failure mode when applicable.

Table 4. Predicate support evidence (auto-generated).

Predicate	Syntax	Yuclid	Newclid 3.0.1	Notes
DIFFERENT	diff a b c ...	No	Yes	Yuclid: unknown statement diff. Newclid works.
MIDPOINT	midp m a b	Yes	Yes	Works.
COLLINEAR	coll a b c ...	Yes	Yes	Works.
N_COLLINEAR	ncoll a b c ...	No	Yes	Yuclid: unknown statement ncoll. Newclid works.
PERPENDICULAR	perp a b c d	Yes	Yes	Works.
N_PERPENDICULAR	nperp a b c d	No	Yes	Yuclid: unknown statement nperp. Newclid works.
PARALLEL	para a b c d	Yes	Yes	Works.
N_PARALLEL	npara a b c d	No	Yes	Yuclid: unknown statement npara. Newclid works.
CIRCLE	circle o a b c	Yes	Yes	Works.
CYCLIC	cyclic a b c d	Yes	Yes	Works.
CONSTANT_ANGLE	aconst a b c d r	Yes	Yes	Works.
A_COMPUTE	acompute a b c d	No	No	Yuclid: unknown statement acompute. Newclid: arithmetic-reasoning limitation.
OBTUSE_ANGLE	obtuse_angle a b c	Yes	Yes	Works.
EQUAL_ANGLES	eqangle a b c d e f g h	Yes	Yes	Works.
CONSTANT_LENGTH	lconst a b l	Yes	Yes	Works.
L_COMPUTE	lcompute a b	No	No	Yuclid: unknown statement lcompute. Newclid: arithmetic-reasoning limitation.
SQUARED_CONSTANT_LENGTH	l2const a b l	Yes	Yes	Works.
CONGRUENT	cong a b c d	Yes	Yes	Works.
CONSTANT_RATIO	rconst a b c d r	Yes	Yes	Works.
R_COMPUTE	rcompute a b c d	N/A	N/A	Listed in documentation, but no accessible minimal example was found. End-to-end behavior could not be evaluated.
SQUARED_CONSTANT_RATIO	r2const a b c d r	Yes	Yes	Works.
EQUAL RATIOS	eqratio a b c d e f g h	Yes	Yes	Works.
SIMTRI_CLOCK	simtri a b c p q r	Yes	Yes	Works.
SIMTRI_REFLECT	simtrir a b c p q r	Yes	Yes	Works.
CONTRI_CLOCK	contri a b c p q r	Yes	Yes	Works.
CONTRI_REFLECT	contrir a b c p q r	Yes	Yes	Works.
SAME_CLOCK	sameclock a b c x y z	Yes	Yes	Works.
SAME_SIDE	sameside a b c x y z	Yes	Yes	Works.

Predicate	Syntax	Yuclid	Newclid 3.0.1	Notes
N_SAME_SIDE	nsameside a b c x y z	Yes	Yes	Works.
PYTHAGOREAN_PREMISES	pythagorean_premises a b c	No	N/A	Documented as internal (for r57); no accessible minimal example.
PYTHAGOREAN_CONCLUSIONS	pythagorean_conclusions a b c	No	N/A	Same as above.
LENGTH_EQUATION	lequation k ₁ a ₁ b ₁ ... k ₂ c ₁ d ₁ ... k	No	No	NotImplementedError: LengthEquationPredicate is not implemented. See Appendix D.
ANGLE_EQUATION	aequation k ₁ a ₁ b ₁ c ₁ d ₁ k ₂ a ₂ b ₂ c ₂ d ₂ ... k	No	No	NotImplementedError: AngleEquationPredicate is not implemented. See Appendix D.

D. Investigation: lequation Predicate Error in Newclid 3.0.1

Context We write geometry problems in JGEX format and solve them with Newclid. The Newclid manual documents a predicate `lequation` ([predicate reference](#)). In Newclid 3.0.1, using `lequation` raises:

```
NotImplementedError: LengthEquationPredicate is not implemented
```

Specification in the manual The manual maps `lequation` to predicate type `LENGTH_EQUATION` with syntax:

```
lequation k1 a1 b1 a2 b2 ... k2 c1 d1 c2 d2 ... k
```

Interpreted as validity of an equation of the form

$$k_1 \cdot |a_1 b_1| \cdot |a_2 b_2| \cdots + k_2 \cdot |c_1 d_1| \cdot |c_2 d_2| \cdots + \cdots = k.$$

Minimal reproducible example The following JGEX input includes a `lequation` goal. For readability, it is shown below with a visual line break, although the actual input is a single line.

```
c a b = between c a b ?
lequation 1/1 a c 1/1 c b -1/1 a b 0/1
```

The following Python snippet triggers the failure during problem setup:

```
import numpy as np
from newclid.jgex.problem_builder import JGEXProblemBuilder
from newclid.api import GeometricSolverBuilder

rng = np.random.default_rng(0)
problem_name = "pla"
problem_string = (
    "c a b = between c a b ? "
    "lequation 1/1 a c 1/1 c b -1/1 a b 0/1"
)

jgex_problem_builder = (
    JGEXProblemBuilder(rng=rng)
    .include_auxiliary_clauses(True)
    .with_problem_from_txt(
        problem_txt=problem_string, problem_name=problem_name
    )
)

problem_setup = jgex_problem_builder.build() # fails here
solver = GeometricSolverBuilder(rng=rng).build(problem_setup)
solver.run()
```

Observed failure Newclid 3.0.1 fails at build time with:

```
NotImplementedError: LengthEquationPredicate is not implemented
```

Root cause in the source code The exception is raised in `newclid/predicates/__init__.py` inside the predicate factory. The factory explicitly refuses to construct a predicate object for `PredicateType.LENGTH_EQUATION`.

Evidence of missing implementation A related class `LengthEquation` exists in `newclid/predicates/equation_length.py`, but a class named `LengthEquationPredicate` does not exist in the installed package. Therefore, `lequation` is not supported end-to-end in Newclid 3.0.1: the build process fails before geometric solving begins.

E. Adoption of the Documentation Recommendations Beyond JGEXDiv

Assessment protocol. To examine whether the documentation issues identified in JGEX are specific to that setting, we reviewed representative public releases from the Lean ecosystem and from recent executable geometry systems. We assess each release against the four components of the executable interface contract introduced in the Introduction. A *Yes* in (i) requires the evaluated formal benchmark instances to be publicly available in runnable form; in (ii), the toolchain and benchmark-relevant dependencies to be pinned sufficiently to reconstruct the evaluation environment; in (iii), benchmark-facing executable evidence that makes the effective supported interface auditable; and in (iv), explicit provenance for nontrivial corrections, reformulations, auxiliary constructions, exclusions, or analogous curation choices. We use *Partial* when the release provides substantive evidence toward a component but does not satisfy all of these conditions, and *Not found* when we did not identify corresponding evidence in the reviewed public release as of August 2026. Within each table, rows are ordered from weaker to stronger adherence to the four-component contract; this ordering reflects only the artifact assessment defined here, not the scientific quality or performance of the benchmark. For every non-*Yes* rating below, the Evidence column states the specific missing artifact or information that motivated the rating.

Interpreting support evidence in Lean. Recommendation (iii) is necessarily system-dependent. For a compact geometry DSL such as JGEX, suitable evidence can take the form of predicate- and constructor-level conformance tests. In Lean, the analogous evidence is executable validation that the released benchmark and evaluation harness work under the stated environment, together with explicit dependency or trust assumptions when these affect what the evaluator accepts. We therefore do not expect a predicate-level support matrix for Lean; rather, we ask whether the effective benchmark interface is made executable and auditable.

Table 5. Adherence to the four components of the proposed executable interface contract in representative Lean benchmark releases. Ratings concern only the completeness of the reviewed public artifacts with respect to our proposed contract; they do not assess the scientific quality, difficulty, or usefulness of the benchmark.

Benchmark	(i)	(ii)	(iii)	(iv)	Evidence and reason for non-Yes ratings
LeanEuclid (Murphy et al., 2024)	Yes	Partial	Partial	Partial	Releases 173 manually formalized geometry problems as a buildable Lean project, together with a <code>lean-toolchain</code> , <code>lake-manifest.json</code> , a <code>Dockerfile</code> , and build instructions. Why not Yes: for (ii), the external SMT environment is only partially pinned: the setup instructions ask users to install the latest versions of Z3 and CVC5. For (iii), the project provides executable build and evaluation infrastructure, but we did not identify a benchmark-facing conformance audit of the System E/SMT interface exercised by the benchmark. For (iv), the release explains that missing diagrammatic information was manually supplied when adapting UniGeo problems, but does not systematically record these additions for every affected instance.
ProofNet (Azerbayev et al., 2023)	Yes	Partial	Partial	Partial	Releases 371 paired informal and Lean theorem statements together with maintenance and evaluation scripts. Why not Yes: for (ii), the canonical release targets Lean 3, which is no longer maintained, while current use relies on separately maintained Lean 4 ports; there is therefore no single canonical version identity spanning these releases. For (iii), the original evaluation scripts support the historical release, but we did not identify a canonical compatibility or validation layer for the current Lean 4 ports. For (iv), paired source material is released, but we did not find a systematic per-instance record of corrections or reformalizations introduced across versions or ports.

Position: Mathematical DSL Benchmarks Need Executable Interface Contracts

Benchmark	(i)	(ii)	(iii)	(iv)	Evidence and reason for non-Yes ratings
Lean Refactor / Arena (Lu et al., 2026; Delta Lab AI, 2026)	Partial	Yes	Yes	Partial	The associated work places unusual emphasis on version-sensitive executability: proof transformations are verified by compilation, and non-Mathlib shortened proofs are recompiled across multiple Lean toolchains to record compatibility. This provides particularly strong evidence for (ii) and (iii). Why not Yes: for (i), although the paper reports the release of a large corpus of verified long-short proof pairs, at the time of our public-artifact review we did not identify a standalone, citable, versioned artifact containing the complete corpus used in the reported experiments; we therefore rate this <i>Partial</i> rather than <i>Not found</i> . For (iv), the work documents corpus construction, filtering, and refactoring strategies, but we did not identify per-instance provenance recording the source and nontrivial curation decisions for each benchmark item.
LeanGeo-Bench (Song et al., 2025)	Yes	Yes	Partial	Partial	Releases 122 benchmark problems in a Lean 4.15 project with a <code>lean-toolchain</code> , <code>lake-manifest.json</code> , build instructions, and an evaluation procedure. Sources and production methods are reported for the benchmark subsets. Why not Yes: for (iii), the released build and evaluation procedure establishes executability, but we did not identify a separate benchmark-facing conformance audit of the formal interface exercised by LeanGeo-Bench. For (iv), curation is described at subset level (e.g., manually written, generated, or autoformalized and checked), but we did not identify per-instance records of corrections, added assumptions, or reformulations.
miniF2F Lean 4 (Zheng et al., 2021)	Yes	Yes	Partial	Partial	Releases Lean 4 formal statements in a pinned Lake project and adds natural-language source docstrings. The release explicitly reports removing known false statements and strengthening many formalizations to better match their English counterparts. Why not Yes: for (iii), compilation against the pinned project provides executable evidence, but we did not identify a separate benchmark-facing audit of the effective supported interface. For (iv), the release documents the correction policy and its motivation, but does not provide a complete per-instance changelog identifying every corrected or strengthened formalization and its rationale.
Formal Conjectures (Firsching et al., 2026)	Yes	Yes	Partial	Yes	Releases a buildable Lean 4 benchmark with pinned project dependencies and an explicit benchmark-versioning policy. Stable snapshots identify both benchmark and Lean versions; correcting a misformalization requires a new immutable benchmark version, and original sources are recorded. Why not Yes: only (iii) is rated Partial. The repository provides a buildable and tested Lean project, but we did not identify a separate benchmark-facing evaluator or conformance artifact that explicitly specifies and tests the effective interface accepted for benchmark solutions.
Lean Eval (Lean Eval Contributors, 2026)	Yes	Yes	Yes	Partial	Releases trusted formal problem statements together with one manifest per problem, generated evaluation workspaces, validators, end-to-end build checks, and external checker dependencies pinned to immutable commits. Why not Yes: only (iv) is rated Partial. Per-problem manifests support fields for <code>source</code> , <code>notes</code> , and <code>informal_solution</code> , but these provenance fields are optional rather than required for every problem. The infrastructure can therefore record curation provenance, but does not enforce it uniformly across the benchmark.

Summary for Lean. The Lean comparison does not support the view that these practices are absent from the ecosystem. On the contrary, recent releases provide several strong positive examples: Formal Conjectures explicitly versions both benchmark content and Lean dependencies, Lean Eval provides a version-pinned executable evaluation contract, and Lean Refactor directly tests compatibility across multiple Lean versions. Adoption is nevertheless uneven: execution environments vary in how completely they are pinned, benchmark-facing support evidence is not universal, and systematic per-instance curation provenance remains less consistent.

Table 6. Adherence to the four components of the proposed executable interface contract in representative executable-geometry releases. Ratings concern only the completeness of the reviewed public artifacts with respect to our proposed contract; they do not assess the scientific quality, difficulty, or usefulness of the benchmark or system.

Benchmark/system	Formal interface	(i)	(ii)	(iii)	(iv)	Evidence and reason for non-Yes ratings
AlphaGeometry2 (Chervonyi et al., 2025)	Expanded Alpha-Geometry DSL	Partial	Not found	Partial	Partial	The paper provides substantial interface-level evidence: it reports 88% coverage of the 2000–2024 IMO geometry problems and identifies important classes that remain outside the language. The public repository releases the DDAR2 symbolic core and executable examples, including examples with manually supplied auxiliary points. Why not Yes: for (i), the public repository reproduces the symbolic DDAR component on selected problems rather than releasing the complete AlphaGeometry2 system and full IMO-AG-50 evaluation package. For (ii), we did not identify a pinned software environment or immutable benchmark version corresponding to the reported results. For (iii), the 88% language-coverage analysis is valuable support evidence, but is not accompanied by an executable conformance suite for the full benchmark-used language fragment. For (iv), the paper discusses formalization limitations and auxiliary points, but we did not identify a complete per-instance record of these decisions for the full evaluation set.
TongGeometry (Zhang et al., 2024)	Custom geometry DSL	Partial	Partial	Partial	Partial	Releases the solver library, tests, installation requirements, an online manual, model code, and generated data, and explicitly notes that its DSL differs from AlphaGeometry’s. Why not Yes: for (i), we did not identify a separately packaged, frozen release of the exact formal benchmark instances used for the reported evaluations. For (ii), the public dependency specification leaves most packages unpinned and we did not identify a benchmark snapshot explicitly tied to the reported results. For (iii), tests and documentation exercise the implementation, but we did not find a systematic conformance audit of the DSL fragment used by the benchmark. For (iv), the release documents the system and data-generation process, but not per-instance reformulations, exclusions, or auxiliary encoding decisions.

Position: Mathematical DSL Benchmarks Need Executable Interface Contracts

Benchmark/system	Formal interface	(i)	(ii)	(iii)	(iv)	Evidence and reason for non-Yes ratings
GenesisGeo (Zhu et al., 2025)	DDARN / Newclid; includes JGEX-AG-231	Yes	Partial	Partial	Partial	Releases executable benchmark files including IMO-AG-30, IMO-95, HAGeo-409, and JGEX-AG-231, together with a dependency lockfile, tests, generated problems with proof traces, and minimal regression checks for individual deduction rules. Why not Yes: for (ii), the dependency lockfile improves reproducibility, but we did not identify an immutable benchmark version or release explicitly tied to the reported evaluation snapshot. For (iii), the rule-level regression tests provide meaningful executable support evidence, but do not constitute a complete audit of the predicate/constructor or DDARN interface exercised by the benchmark. For (iv), the generation pipeline and proof traces are released, but we did not identify systematic per-instance documentation of benchmark reformulations or auxiliary-construction choices.
JGEXDiv (ours)	JGEX / Newclid	Yes	Yes	Yes	Yes	Releases the formal benchmark instances and validator scripts, pins Newclid v3.0.1, provides executable predicate-level support evidence, and records nontrivial reformulations, helper constructions, exclusions, and selected alternative encodings. JGEXDiv is included as the reference case from which the proposed contract is derived rather than as an independent assessment.

Summary for executable geometry. The geometry comparison similarly reveals partial but nontrivial adoption of the proposed contract. GenesisGeo releases executable benchmarks and rule-level regression checks; TongGeometry releases its solver, DSL implementation, tests, manual, and generated data; and AlphaGeometry2 explicitly reports the coverage and limitations of its expanded formal language. The main gap is therefore not an absence of documentation or executable artifacts, but that executable instances, versioned environments, interface-support evidence, and formalization provenance are not consistently released together as a single auditable contract.